UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

DOCENTE:

Ing. Ma. Carmelo Estuardo Mayen Monterroso

CURSO:

Inteligencia Artificial

SEMESTRE:

Noveno

TEMA:

Búsqueda Primero En Anchura y Búsqueda A*

ESTUDIANTES:	**CARNÉ:**
López Morales Gelen Dayanna	1790-21-14904
Lorenzana León Jaqueline Natalia	1790-22-13193
Rivera Pérez Javier José Luis	1790-22-10552
Rodríguez Arévalo Karla Waleska	1790-22-9763
Ojeda Esquite Bagner Francisco	1790-18-2522

FECHA DE ENTREGA: sábado 21 de marzo de 2026

BÚSQUEDA PRIMERO EN ANCHURA

1. Definición y Contexto:

La Búsqueda Primero en Anchura (BFS, del inglés Breadth-First Search) es un algoritmo de búsqueda no informada utilizado para recorrer o buscar elementos en estructuras de datos de tipo árbol o grafo. Su estrategia consiste en expandir el nodo raíz y, a continuación, expandir todos sus sucesores antes de avanzar al siguiente nivel de profundidad.

En el contexto de la Inteligencia Artificial, BFS pertenece a la categoría de búsqueda ciega, es decir, no utiliza información heurística para guiar la exploración. Esto la distingue de algoritmos como A* o la búsqueda voraz, que sí incorporan estimaciones del costo hacia el objetivo.

En términos formales, el algoritmo visita todos los nodos a distancia d de la raíz antes de pasar a los nodos a distancia d+1, garantizando así que el primer camino encontrado hacia el objetivo es el de menor longitud (en número de aristas), en grafos no ponderados.

BFS es ampliamente estudiado porque combina simplicidad con aplicaciones prácticas en redes, inteligencia artificial y resolución de problemas. Su garantía de encontrar el camino de menor número de aristas lo convierte en una herramienta indispensable en el diseño de soluciones óptimas.

2. Fundamento: La Cola FIFO:

El elemento central que distingue a BFS de otros algoritmos de búsqueda es el uso de una estructura de datos de tipo Cola (Queue), que opera bajo el principio FIFO (First In, First Out — el primero en entrar es el primero en salir). Esta propiedad garantiza que los nodos sean expandidos en el mismo orden en que fueron descubiertos, asegurando la exploración nivel por nivel.

El algoritmo mantiene además dos listas auxiliares:

- Lista abierta (cola): nodos descubiertos pero aún no expandidos.
- Lista cerrada (visitados): nodos ya expandidos, para evitar ciclos y visitas repetidas.

3. Mecanismo de Funcionamiento:

Dado un vértice fuente s, BFS sistemáticamente explora los vértices del grafo G para descubrir todos los vértices alcanzables desde s. El algoritmo llega a los nodos de distancia k únicamente después de haber alcanzado todos los nodos a distancia k-1, expandiendo uniformemente la frontera entre lo descubierto y lo no descubierto.

El algoritmo BFS puede describirse formalmente en los siguientes pasos:

1. Insertar el nodo raíz en la cola y marcarlo como visitado.
2. Extraer el primer elemento de la cola (orden FIFO).
3. Si el nodo extraído es el objetivo, retornar la solución y terminar.
4. Si no es el objetivo, generar todos sus nodos vecinos no visitados.
5. Marcar cada vecino como visitado, registrar su nodo padre, e insertarlo al final de la cola.
6. Repetir desde el paso 2 hasta que la cola esté vacía (sin solución) o se halle el objetivo.

3.1. Pseudocódigo:

Pseudocódigo formal del algoritmo:

```
función BFS(Grafo G, nodo_inicio s, nodo_meta g):
   cola    ← cola vacía
   visitó ← Conjuntovac ío
   padre   ← diccionario vacío

   encolar(cola, s)
   agregar(visitados, s)

   mientras cola no esté vacía:
      nodo ← desencolar(cola)        // FIFO: extrae el frente

      si node == g:
         Regresar reconstruir_camino(Padre, S, G)

      para cada vecino v de nodo en G:
         si v no está en visitados:
            agregar(visitados, v)
 nodo padre[v] ←
            encolar(cola, v)
   retornar 'Sin solución encontrada'
```

4. Formulación Formal como Problema de IA

En el marco de la Inteligencia Artificial, todo problema de búsqueda puede formularse con los siguientes componentes. Aplicados al contexto de BFS, quedan definidos así:

COMPONENTE	DESCRIPCIÓN EN BFS
Estado inicial	Nodo o configuración desde la cual comienza la búsqueda (nodo raíz).
Espacio de estados	Conjunto de todos los nodos o configuraciones alcanzables desde el estado inicial.
Función de transición	Regla que define los estados sucesores dado un estado actual (vecinos en el grafo).
Prueba de objetivo	Condición que verifica si el nodo actual es el estado meta buscado.
Costo de camino	Número de aristas recorridas. BFS minimiza este valor en grafos no ponderados.

5. Propiedades Técnicas del Algoritmo

PROPIEDAD	VALOR / DESCRIPCIÓN
Completitud	Sí — Si existe una solución, BFS la encontrará (factor de ramificación finito).
Optimalidad	Sí — Encuentra el camino más corto en grafos no ponderados (costo uniforme por paso).
Complejidad temporal	O(V+E) donde V = número de vértices y E = número de aristas. Equivalente a O(b^(d+1)) expresado en términos del factor de ramificación b y profundidad d.

Complejidad espacial	O(V) — almacena los nodos visitados. En términos del factor de ramificación: O(b^d). Principal desventaja en grafos grandes.
Tipo de búsqueda	No informada (ciega) — no utiliza heurísticas para guiar la exploración.

6. BFS vs. Búsqueda Primero en Profundidad (DFS)

Ambos algoritmos son fundamentales en la teoría de grafos y la Inteligencia Artificial. Sus diferencias clave se resumen en la siguiente tabla comparativa:

CARACTERÍSTICA	FSO	DFS
Estructura de datos	Cola (FIFO)	Pila (LIFO)
Orden de exploración	Nivel por nivel (anchura)	Rama por rama (profundidad)
Garantía de optimalidad	Sí (grafos no ponderados)	No
Complejidad espacial	O(V) — puede ser alta	O(b·m) — generalmente menor
Grafos ponderados	No recomendado — usar Dijkstra	Tampoco es óptimo
Mejor uso	Rutas más cortas, grafos con solución cerca de la raíz	Exploración de grafos profundos

7. Ejemplo Gráfico: Recorrido BFS en un Árbol

Considérese el siguiente árbol con nodo raíz A y tres niveles de profundidad. BFS lo recorre expandiendo todos los nodos de cada nivel antes de descender al siguiente.

Estructura del árbol de ejemplo:

```
      [A]        ← Nivel 0 (Raíz)
    / | \
  [B] [C] [D]    ← Nivel 1
  / \  |  / \
[E][F] [G] [H][I]  ← Nivel 2
```

Traza de ejecución paso a paso:

Paso	Nodo extraído	Estado de la cola	Acción
0	—	[A]	Inicialización: raíz A en cola
1	A	[B, C, D]	Se extraen vecinos de A → se encolan B, C, D
2	B	[C, D, E, F]	Se extraen vecinos de B → se encolan E, F
3	C	[D, E, F, G]	Se extrae vecino de C → se encola G
4	D	[MI, FA, SOL, H, I]	Se extraen vecinos de D → se encolan H, I
5	E	[F, G, H, I]	E es hoja — sin nuevos nodos
6	F	[G, H, I]	F es hoja — sin nuevos nodos
7	G	[H, I]	G es hoja — sin nuevos nodos
8	H	[I]	H es hoja — sin nuevos nodos
9	I	[ ]	I es hoja — cola vacía. Recorrido completo

Orden de visita resultante: A → B → C → D → E → F → G → H → I

8. Ejemplo de Problema Aplicado: El Laberinto

8.1. Planteamiento del Problema

Un robot (agente inteligente) se encuentra en la entrada de un laberinto representado por una cuadrícula de N x M celdas. Algunas celdas son paredes (obstáculos) y otras son pasillos libres. El robot puede moverse en cuatro direcciones: arriba, abajo, izquierda, derecha. El objetivo es encontrar la ruta más corta desde la celda de entrada hasta la celda de salida.

Formulación del Problema en Términos de IA

- Estado inicial: coordenadas (fila, columna) de la entrada del laberinto.
- Espacio de estados: todas las celdas libres (no paredes) del laberinto.
- Función de transición: moverse a una celda adyacente libre (arriba, abajo, izquierda, derecha).
- Prueba de objetivo: verificar si la celda actual es la celda de salida.
- Costo de camino: número de movimientos realizados (cada movimiento cuesta 1 unidad).

Por qué BFS es la Elección Correcta

En este laberinto, cada movimiento tiene el mismo costo (1 unidad). Por lo tanto:

- BFS garantiza encontrar el nodo objetivo en el nivel de profundidad mínimo posible.
- Al explorar 'en ondas' concéntricas desde el punto de inicio, la primera vez que la búsqueda alcanza la salida, el camino recorrido es matemáticamente el más corto.
- Algoritmos como DFS podrían encontrar una solución, pero no necesariamente la de menor longitud.

Representación del Laberinto

Laberinto (5x6): S = Inicio G = Meta # = Pared . = Libre

Columnas:	0	1	2	3	4	5
Fila 0:	[S]	[.]	[#]	[.]	[.]	[.]
Fila 1:	[#]	[.]	[#]	[.]	[#]	[.]
Fila 2:	[.]	[.]	[.]	[.]	[#]	[.]
Fila 3:	[.]	[#]	[#]	[.]	[.]	[.]
Fila 4:	[.]	[.]	[.]	[#]	[.]	[G]

Traza de BFS en el Laberinto

El algoritmo expande celdas en orden de distancia desde S (0,0). La solución encontrada es la ruta más corta:

Camino más corto encontrado por BFS:
(0,0) → (0,1) → (1,1) → (2,1) → (2,2) → (2,3) → (3,3) → (3,4) → (3,5) → (4,5)

Longitud del camino: 9 movimientos (garantizado mínimo por BFS)

Nótese que BFS 'irradia' desde S como una ola de agua: primero explora todas las celdas a distancia 1, luego a distancia 2, y así sucesivamente. La primera vez que alcanza G, el camino acumulado es el óptimo.

8.2. El Caballo de Ajedrez

Encontrar la menor cantidad de movimientos para que un caballo de ajedrez vaya desde una casilla inicial hasta una casilla objetivo en un tablero de 8×8.

Formulación del problema:

- Cada casilla del tablero es un nodo.
- Cada movimiento válido del caballo (hasta 8 posibles en forma de 'L') es una arista.
- Se utiliza BFS porque cada movimiento tiene el mismo costo (1 movimiento) y se busca el camino con menos movimientos.

Solución conceptual con BFS:

1. Iniciar la cola con la casilla inicial y distancia 0.
2. Mientras la cola no esté vacía: extraer una casilla; si es el objetivo, retornar la distancia; de lo contrario, generar todos los movimientos válidos del caballo y encolar los no visitados con distancia +1.
3. BFS garantiza que la primera vez que se alcanza el objetivo es con la distancia mínima.

Resultado:

Inicio: (0, 0)
Objetivo: (7, 7)

BFS encuentra la distancia minima: 6 movimientos.
(BFS garantiza que no existe ninguna ruta con menos de 6 movimientos.)

9. Aplicaciones Prácticas de BFS

- **Redes de computadoras:** Algoritmos de enrutamiento que buscan la ruta con menor número de saltos entre nodos de una red.
- **Redes sociales:** Determinación del grado de separación entre usuarios (el problema de los '6 grados de separación').
- **Sistemas de navegación GPS:** Búsqueda de rutas más cortas en mapas representados como grafos de intersecciones.
- **Resolución de puzzles:** Laberintos, rompecabezas deslizantes (15-puzzle), Cubo de Rubik con heurísticas.
- **Inteligencia Artificial en juegos:** Exploración de estados del tablero para encontrar movimientos ganadores en el menor número de turnos.
- **Modelos de decisión (ML):** BFS es base de los árboles de decisión en algoritmos de aprendizaje automático.
- **Verificación de conectividad:** Comprobar si todos los nodos de un grafo son accesibles desde un nodo fuente, útil en diagnóstico de redes.
- **Algoritmos avanzados:** BFS sirve de base para variantes como la búsqueda bidireccional y, combinado con otros métodos, para búsqueda en grafos ponderados.

10. Limitaciones del Algoritmo

A pesar de sus ventajas, BFS presenta limitaciones importantes a considerar al momento de seleccionarlo como estrategia de búsqueda:

- **Consumo de memoria:** No es eficiente en grafos muy grandes, ya que requiere memoria proporcional al número de nodos almacenados en la frontera.
- **Grafos ponderados:** No distingue el peso de las aristas, por lo que no garantiza el camino de menor costo cuando las aristas tienen pesos distintos. En esos casos se recomienda el algoritmo de Dijkstra (pesos positivos) o Bellman-Ford (pesos negativos).
- **Factor de ramificación alto:** En grafos con muchos vecinos por nodo, la cola puede crecer exponencialmente, haciendo inviable el algoritmo en la práctica.
- **Sin heurística:** No aprovecha información adicional del problema (heurísticas). Para problemas donde se dispone de una estimación de la distancia al objetivo, algoritmos como A* son más eficientes.

R

BÚSQUEDA A*

1. Definición y Contexto

La Búsqueda A* (pronunciado 'A estrella' o 'A asterisco') es un algoritmo de búsqueda informada utilizado ampliamente en informática, inteligencia artificial y teoría de grafos. Fue presentado por primera vez en 1968 por Peter E. Hart, Nils J. Nilsson y Bertram Raphael del Instituto de Investigación de Stanford (SRI International). Se trata de una extensión del algoritmo de Dijkstra (1959), al que incorpora una función heurística para guiar la búsqueda de manera más eficiente.

Su función principal es encontrar, siempre y cuando se cumplan determinadas condiciones, el camino de menor costo entre un nodo origen y un nodo objetivo en un grafo ponderado. A diferencia de los algoritmos de búsqueda no informada (como BFS o DFS), A* utiliza información adicional sobre el problema para reducir el número de nodos explorados y llegar a la solución de manera más rápida.

A* destaca por ser la forma más ampliamente conocida de la búsqueda 'primero el mejor' (Best-First Search), combinando lo mejor de dos mundos: la optimalidad de la búsqueda de costo uniforme y la eficiencia de la búsqueda voraz (Greedy Best-First Search).

2. La Función de Evaluación: f(n) = g(n) + h(n)

El elemento central y distintivo de A* es su función de evaluación f(n), que combina dos componentes para determinar qué nodo explorar a continuación. Esta función permite al algoritmo tomar decisiones informadas, priorizando los caminos más prometedores:

COMPONENTE	NOMBRE	DESCRIPCIÓN
g(n)	Costo real acumulado	El costo real del camino recorrido desde el nodo inicial hasta el nodo n. Es el costo exacto y conocido hasta el momento.

h(n)	Función heurística	Estimación del costo restante desde el nodo n hasta el nodo objetivo. Es una estimación, no el valor exacto. Debe ser admisible.
f(n) = g(n) + h(n)	Costo total estimado	La suma de ambos. A* siempre expande el nodo con el menor f(n) de la lista abierta, garantizando un equilibrio entre costo conocido y distancia estimada.

Esta función de evaluación es la piedra angular del proceso de toma de decisiones del algoritmo. El algoritmo selecciona continuamente el nodo con el valor f(n) más bajo de la lista abierta, lo evalúa y lo mueve a la lista cerrada hasta encontrar el nodo meta.

2.1. Casos especiales de la función

Dependiendo de los valores de g(n) y h(n), A* puede comportarse como otros algoritmos conocidos:

- **Cuando h(n) = 0:** Si h(n) = 0 para todo nodo, A* se convierte en una Búsqueda de Costo Uniforme (Dijkstra), ya que solo considera el costo real.
- **Cuando g(n) = 0:** Si g(n) = 0 para todo nodo, A* se convierte en una Búsqueda Voraz (Greedy), guiándose solo por la heurística.
- **Heurística perfecta:** Con una heurística perfecta h(n) = h*(n) (costo real exacto), A* va directamente al objetivo sin explorar nodos innecesarios.

3. La Función Heurística h(n)

La función heurística h(n) es el componente que convierte a A* en un algoritmo informado. Proporciona una estimación del costo restante desde un nodo n hasta el nodo objetivo, aprovechando información específica del problema para guiar la búsqueda.

3.1. Condiciones que debe cumplir la heurística

- **Admisibilidad:** h(n) ≤ h*(n) para todo nodo n, donde h*(n) es el costo real al objetivo. La heurística nunca debe sobreestimar el costo real. Si esta condición no se cumple, el algoritmo pasa a denominarse simplemente 'A' y pierde la garantía de optimalidad.

- **Consistencia (o monotonía):** Para todo nodo n y cualquier sucesor y: $h(n) \leq costo(n \rightarrow y) + h(y)$. El costo estimado desde n no puede ser mayor que el costo de ir al sucesor más el costo estimado desde el sucesor. Esta condición garantiza que A* nunca necesita re-explorar nodos.
-

3.2. Heurísticas comunes según el tipo de problema

TIPO DE PROBLEMA	HEURÍSTICA UTILIZADA	DESCRIPCIÓN
Cuadrícula / Mapa 2D (mov. en 4 dir.)	Distancia Manhattan	$\|x1 - x2\| + \|y1 - y2\|$ — suma de diferencias en filas y columnas.
Cuadrícula / Mapa 2D (mov. en 8 dir.)	Distancia Chebyshev	$max(\|x1-x2\|, \|y1-y2\|)$ — máximo entre diferencia horizontal y vertical.
Espacio abierto / Coordenadas reales	Distancia Euclidiana	$sqrt((x1-x2)^2 + (y1-y2)^2)$ — distancia en línea recta.
Puzzle / Problema de estados	Distancia Manhattan por pieza	Suma de las distancias Manhattan de cada pieza a su posición correcta.
Grafo sin coordenadas	$h(n) = 0$	Heurística nula — degenera en búsqueda de costo uniforme (Dijkstra).

4. Estructuras de Datos Auxiliares

A* mantiene dos estructuras de datos auxiliares durante su ejecución, que permiten gestionar eficientemente qué nodos han sido explorados y cuáles están pendientes:

- **Lista Abierta (Open Set):** Conjunto de nodos descubiertos, pero aún no expandidos. Se implementa como una Cola de Prioridad (min-heap) ordenada por el valor f(n) de cada nodo. El nodo con menor f(n) siempre está al frente, listo para ser expandido. En el inicio, contiene únicamente el nodo raíz.
- **Lista Cerrada (Closed Set):** Conjunto de nodos ya expandidos (visitados). Evita que el algoritmo procese el mismo nodo más de una vez, previniendo ciclos y redundancias. Una vez que un nodo pasa a cerrados, no necesita ser re-evaluado si la heurística es consistente.

5. Mecanismo de Funcionamiento

En cada paso del algoritmo, se expande el nodo que esté primero en la lista abierta (el de menor f(n)). Si no es el nodo objetivo, se calculan los valores f(n) de todos sus sucesores, se insertan en la lista abierta y el nodo evaluado pasa a la lista cerrada. El algoritmo es una combinación entre búsqueda en anchura (guiada por g(n)) y búsqueda en profundidad (guiada por h(n)).

1. Inicializar la lista abierta con el nodo inicial. Calcular f(inicio) = g(inicio) + h(inicio) = 0 + h(inicio).
2. Si la lista abierta está vacía, no existe solución. Terminar.
3. Extraer de la lista abierta el nodo n con menor f(n).
4. Si n es el nodo objetivo, reconstruir el camino y retornar la solución.
5. Mover n a la lista cerrada (marcarlo como expandido).
6. Para cada sucesor y de n: calcular g(y) = g(n) + costo(n→y) y f(y) = g(y) + h(y). Si y ya está en cerrados con un costo menor o igual, ignorarlo. Si y está en abiertos con un f(y) mayor, actualizarlo. Si no está en ninguna lista, agregarlo a abiertos.
7. Repetir desde el paso 2.

5.1. Pseudocódigo formal

```
función A_ESTRELLA(Grafo G, nodo_inicio s, nodo_meta g, heurística
h):
    abiertos  ← Cola de prioridad con s  (f(s) = h(s))
    cerrados  ← Conjunto vacío
    g_costo   ← { s: 0 }          // costo real desde inicio
    padre     ← { s: None }        // para reconstruir el camino

    mientras abiertos no esté vacío:
        n ← extraer_minimo(abiertos)    // nodo con menor f(n)

        si n == g:
            retornar reconstruir_camino(padre, s, g)

        agregar(cerrados, n)

        para cada sucesor y de n en G:
            si y en cerrados: continuar

            g_nuevo ← g_costo[n] + costo(n, y)

            si y no está en abiertos o g_nuevo < g_costo[y]:
                g_costo[y] ← g_nuevo
```

```
            f_y        ← g_nuevo + h(y)
            padre[y]   ← n
            insertar_o_actualizar(abiertos, y, f_y)

    retornar 'Sin solución encontrada'
```

6. Formulación Formal como Problema de IA

En el marco de la Inteligencia Artificial, A* se formula con los siguientes componentes:

COMPONENTE	DESCRIPCIÓN EN A*
Estado inicial	Nodo de origen desde el cual comienza la búsqueda.
Espacio de estados	Todos los nodos del grafo ponderado alcanzables desde el origen.
Función de transición	Regla que define los nodos sucesores y el costo de cada transición.
Prueba de objetivo	Verificar si el nodo actual es el nodo meta buscado.
Función de costo	f(n) = g(n) + h(n). A* minimiza este valor para cada nodo expandido.
Función heurística	h(n): estimación admisible del costo restante hacia el objetivo.

7. Propiedades Técnicas del Algoritmo

PROPIEDAD	VALOR / DESCRIPCIÓN
Tipo de búsqueda	Informada (heurística) — usa información del dominio para guiar la exploración.
Completitud	Sí — si existe una solución, A* siempre la encontrará (con factor de ramificación finito).
Optimalidad	Sí — garantizada si la heurística h(n) es admisible (no sobreestima el costo real).
Complejidad temporal	O(b^d) en el peor caso. Mejora significativamente con una buena heurística, pudiendo llegar a tiempo lineal con una heurística perfecta.

Complejidad espacial	O(b^d) — su mayor debilidad. Debe almacenar todos los nodos posibles en memoria. Variantes como IDA* y SMA* reducen este consumo.
Estructura de datos	Cola de prioridad (min-heap) para la lista abierta, ordenada por f(n).
Año de creación	1968 — Hart, Nilsson y Raphael, SRI International.

8. Comparativa: A* vs BFS vs Dijkstra vs Búsqueda Voraz

A* se posiciona entre los algoritmos no informados (BFS, Dijkstra) y la búsqueda voraz, combinando las ventajas de ambos enfoques:

CARACTERÍSTICA	BFS	DIJKSTRA	VORAZ (GREEDY)	A*
Tipo	No informada	No informada	Informada	Informada
Usa heurística	No	No	Sí (solo h(n))	Sí (f=g+h)
Considera costo	No	Sí	No	Sí
Completitud	Sí	Sí	No siempre	Sí
Optimalidad	Sí (no pond.)	Sí	No	Sí (h admisible)
Eficiencia	Baja	Media	Alta (no óptima)	Alta y óptima
Estructura datos	Cola FIFO	Cola prioridad	Cola prioridad	Cola prioridad

9. Ejemplo Gráfico: A* en un Grafo Ponderado

Se presenta un grafo ponderado donde se busca el camino de menor costo desde el nodo S (inicio) hasta el nodo G (meta). Los números en las aristas representan el costo real de cada trayecto. La heurística h(n) utilizada es una estimación de la distancia restante al objetivo.

9.1. Estructura del grafo y valores heurísticos

Grafo (costos en aristas): Valores heurísticos h(n):

```
      S                    h(S) = 7
     / \                   h(A) = 6
   1/   \4                   h(B) = 2
   /     \                 h(C) = 1
  A       B                  h(D) = 3
 / \     / \                 h(E) = 0  (es el objetivo G)
2/  \3  1/  \5                 h(G) = 0
/    \ /    \
C     D     G
 \   /
 2\ /3
   E
```

9.2. Traza de ejecución de A*

En cada paso se muestra: nodo expandido, su g(n), h(n), f(n) y el estado de la lista abierta.

Paso	Nodo	g(n)	h(n)	f(n)	Acción
1	S	0	7	7	Inicio. Se agregan A (f=1+6=7) y B (f=4+2=6) a abiertos.
2	B	4	2	6	B tiene menor f. Se expanden sus vecinos: D (f=4+1+3=8), G (f=4+5+0=9).
3	A	1	6	7	A expandido. Vecinos: C (f=1+2+1=4), D (f=1+3+3=7, mejor que 8).
4	C	3	1	4	C tiene menor f. Sin nuevos vecinos útiles.

5	D	4	3	7	D expandido (costo via A=4). Vecino E (f=4+3+0=7).
6	E/G	7	0	7	E=G es el objetivo. Solución encontrada con costo total = 7.

Camino óptimo encontrado: S → A → D → G con costo total = 1 + 3 + 3 = 7

Nótese que A no exploró todos los nodos del grafo. Gracias a la heurística, descartó el camino S → B → G (costo = 9) y encontró la solución óptima explorando solo los nodos más prometedores.*

10. Ejemplo de Problema Aplicado: Navegación GPS entre Ciudades

10.1. Planteamiento del Problema

Un sistema de navegación GPS debe encontrar la ruta de menor distancia (en km) entre dos ciudades en un mapa representado como un grafo ponderado. Cada ciudad es un nodo y cada carretera es una arista con su distancia en km como costo. La heurística utilizada es la distancia en línea recta (distancia euclidiana) entre cada ciudad y el destino.

10.2. Formulación del Problema

COMPONENTE DE IA	VALOR EN ESTE PROBLEMA
Estado inicial	Ciudad de origen (ej. Ciudad A)
Espacio de estados	Todas las ciudades del mapa conectadas por carreteras
Función de transición	Moverse por una carretera a una ciudad adyacente (con su costo en km)
Prueba de objetivo	¿La ciudad actual es el destino?
g(n)	Distancia real acumulada en km desde el origen hasta n
h(n)	Distancia en línea recta desde n hasta el destino (siempre ≤ ruta real)
f(n)	g(n) + h(n): estimación del costo total del camino pasando por n

10.3. Por qué A* es la elección correcta

- Las carreteras tienen costos distintos (grafos ponderados), por lo que BFS no garantiza optimalidad.
- La distancia en línea recta es siempre admisible (nunca supera la distancia real por carretera).
- A* reduce drásticamente los nodos explorados al priorizar las ciudades que se acercan al destino.
- Es el algoritmo base de los sistemas GPS modernos (Google Maps, Waze) por su combinación de optimalidad y eficiencia.

10.4. Representación del mapa

Mapa de ciudades (distancias en km en aristas):
h(n) = distancia en linea recta al destino (Ciudad G)

```
[A] --80km-- [B] --40km-- [C]
 |           |           |
60km        35km        50km
 |           |           |
[D] --55km-- [E] --30km-- [G] <- META
 |
90km
 |
[F]
```

Heuristicas h(n) hacia G:
h(A)=140, h(B)=80, h(C)=50, h(D)=100, h(E)=30, h(F)=180, h(G)=0

10.5. Ejecución de A*

A* prioriza los nodos con menor f(n). La traza simplificada:

8. f(A)=0+140=140 → se exploran vecinos B (f=80+80=160) y D (f=60+100=160).
9. Se expande B (f=160): vecino C (f=80+40+50=170), E (f=80+35+30=145).
10. Se expande E (f=145): vecino G (f=115+30+0=145).
11. Se expande G: es el objetivo. Solucion encontrada.

Ruta optima: A -> B -> E -> G
Costo total: 80 + 35 + 30 = 145 km
A* descarto el camino A->D->E->G (costo = 60+55+30 = 145 km, igual)
y A->D->F (costo > 145 km), explorando solo los nodos necesarios.

11. Ejemplo Adicional: El Puzzle de 8 Piezas

El puzzle de 8 piezas es un problema clásico de IA donde se tiene un tablero de 3×3 con 8 fichas numeradas y un espacio vacío. El objetivo es llegar a una configuración final (estado meta) deslizando fichas al espacio vacío, minimizando el número de movimientos.

11.1. Formulación del problema

- Estado inicial: cualquier configuración válida de las 8 fichas.
- Estado meta: las fichas en orden 1-2-3 / 4-5-6 / 7-8-_ (espacio vacío al final).
- Función de transición: deslizar una ficha adyacente al espacio vacío (hasta 4 movimientos).
- Costo g(n): número de movimientos realizados.
- Heurística h(n): Distancia Manhattan — suma de las distancias de cada ficha a su posición correcta.

11.2. Ejemplo de heurística Manhattan

```
Estado actual:      Estado meta:
+---+---+---+       +---+---+---+
| 2 | 8 | 3 |       | 1 | 2 | 3 |
+---+---+---+       +---+---+---+
| 1 | 6 | 4 |       | 4 | 5 | 6 |
+---+---+---+       +---+---+---+
| 7 |   | 5 |       | 7 | 8 |   |
+---+---+---+       +---+---+---+
```

Calculo de h(n) = Distancia Manhattan:
Ficha 1: esta en (1,0), debe ir a (0,0) -> dist = 1
Ficha 2: esta en (0,0), debe ir a (0,1) -> dist = 1
Ficha 3: ya en su lugar -> dist = 0
Ficha 4: ya en su lugar -> dist = 0
Ficha 5: esta en (2,2), debe ir a (1,1) -> dist = 2
Ficha 6: esta en (1,1), debe ir a (1,2) -> dist = 1
Ficha 8: esta en (0,1), debe ir a (2,1) -> dist = 2

h(n) = 1+1+0+0+2+1+2 = 7 (son como minimo 7 movimientos)
Esta heuristica es admisible: nunca sobreestima los movimientos reales.

A con la heurística Manhattan resuelve el puzzle de 8 piezas de manera óptima, explorando significativamente menos estados que BFS (que no tiene heurística) o DFS (que no garantiza optimalidad).*

12. Variantes del Algoritmo A*

Debido a la limitación de memoria de A*, se han desarrollado variantes que mantienen sus propiedades de optimalidad y completitud con menor consumo de recursos:

VARIANTE	CARACTERÍSTICA PRINCIPAL	USO RECOMENDADO
IDA* (A* iterativo por profundidad)	Usa profundización iterativa con límite de costo f. Memoria O(d).	Cuando la memoria es limitada y el espacio de búsqueda es muy grande.
SMA* (A* con memoria acotada)	Usa toda la memoria disponible. Descarta nodos de mayor costo cuando se llena.	Entornos con memoria limitada pero predecible.
RTA* (A* en tiempo real)	Toma decisiones locales rápidas. No garantiza optimalidad global.	Agentes que deben actuar en tiempo real (videojuegos, robótica).
Weighted A* (A* ponderado)	Usa $f(n) = g(n) + w \cdot h(n)$ con $w > 1$. Más rápido pero solución subóptima.	Cuando se prioriza velocidad sobre optimalidad (máx. w veces el óptimo).
Bidirectional A*	Búsqueda simultánea desde origen y destino. Se detiene cuando se encuentran.	Grafos muy grandes con origen y destino conocidos.

13. Aplicaciones Prácticas de A*

- **Sistemas GPS y navegación:** A* es el algoritmo base de Google Maps, Waze y sistemas de navegación para encontrar la ruta más corta entre dos puntos en mapas ponderados.
- **Videojuegos (pathfinding):** Es el estándar de la industria para el movimiento de personajes y NPCs en mundos 2D y 3D, desde juegos de estrategia hasta RPGs.
- **Robótica y drones:** Planificación de trayectorias para robots móviles que deben navegar evitando obstáculos en entornos desconocidos.
- **Inteligencia artificial en juegos de tablero:** Resolución de puzzles como el 8-puzzle, 15-puzzle y laberintos complejos con garantía de solución óptima.
- **Redes de telecomunicaciones:** Enrutamiento de paquetes de datos por la ruta de menor latencia o costo en redes de computadoras.

- **Planificación en IA:** Sistemas de planificación automática donde A* busca secuencias de acciones que llevan de un estado inicial a un estado objetivo.
- **Bioinformática:** Alineación de secuencias de ADN/proteínas y búsqueda de caminos en grafos biológicos.

14. Limitaciones del Algoritmo A*

- **Consumo de memoria exponencial:** Debe almacenar todos los nodos posibles en memoria, lo que puede ser exponencial con respecto al tamaño del problema. Es la principal razón del uso de variantes como IDA* o SMA*.
- **Dependencia de la heurística:** Si la heurística sobreestima el costo real, A* pierde la garantía de optimalidad y pasa a denominarse simplemente 'A'. Diseñar una buena heurística requiere conocimiento profundo del dominio.
- **Ineficiencia con heurística pobre:** En el peor caso (heurística h(n) = 0), A* degenera en Dijkstra, con complejidad O(b^d), lo que lo hace inviable para espacios de búsqueda muy grandes.
- **Grafos con pesos negativos:** Para grafos con pesos negativos en las aristas, A* no funciona correctamente. En esos casos se debe utilizar el algoritmo de Bellman-Ford.
- **Tiempo real limitado:** No es la mejor opción cuando se necesitan respuestas en tiempo real con recursos muy limitados. Variantes como RTA* están diseñadas para esos escenarios.

CONCLUSIÓN

El estudio de los algoritmos de búsqueda constituye uno de los pilares fundamentales de la Inteligencia Artificial, y la Búsqueda Primero en Anchura (BFS) y la Búsqueda A* representan dos puntos esenciales en la evolución de esta disciplina: el primero como base sólida de la búsqueda no informada, y el segundo como referente de la búsqueda heurística informada.

BFS establece los cimientos conceptuales con su enfoque sistemático de exploración nivel por nivel, sustentado en una cola FIFO, garantizando completitud y optimalidad en grafos no ponderados. Su fortaleza radica en la certeza matemática de encontrar siempre el camino de menor número de pasos cuando todas las transiciones tienen el mismo costo. Sin embargo, su complejidad espacial O(b^d) lo convierte en una solución limitada frente a problemas de gran escala o alto factor de ramificación.

Es precisamente en ese punto donde A* toma el relevo. Al incorporar la función de evaluación f(n) = g(n) + h(n), supera las limitaciones de BFS al considerar simultáneamente el costo acumulado real g(n) y una estimación heurística del costo restante h(n). Esta combinación le permite explorar únicamente los caminos más prometedores, reduciendo drásticamente el espacio de búsqueda sin sacrificar la optimalidad, siempre que la heurística sea admisible.

Ambos algoritmos comparten propiedades esenciales: son completos y óptimos bajo sus respectivas condiciones. La diferencia radica en la información disponible: BFS opera a ciegas con igual eficacia en cualquier grafo no ponderado, mientras que A* aprovecha el conocimiento del dominio para ser significativamente más eficiente en grafos ponderados y problemas complejos.

En conjunto, BFS y A* ilustran con claridad la evolución natural del pensamiento algorítmico en IA: de la exploración exhaustiva y sistemática hacia la búsqueda guiada e inteligente. Comprender ambos enfoques, sus fundamentos, sus limitaciones y su relación es, por tanto, un prerrequisito esencial para abordar con solidez los métodos de optimización y planificación en espacios de estados complejos que caracterizan a la Inteligencia Artificial moderna.

Link GitHub

https://github.com/Javiii3er/visualizador-ia-bfs-astar.git